

Instrucciones de uso

Instalación, aplicación de revisión y artefactos de Wolf-Rayet Search

1. Contenido de la entrega

El repositorio contiene el código reproducible, configuraciones, pruebas, documentación, figuras públicas y notebooks compactos. Las bases DuckDB y los modelos seleccionados se publicaron en la [GitHub Release de esta entrega](#), dentro de `wolf_rayet_search_review_bundle.zip`. El bundle se extrae en la raíz del repositorio y conserva las rutas internas. Esta separación evita incorporar archivos binarios grandes al historial de Git.

El bundle incluye:

- `training_history.duckdb`, con métricas y predicciones de las 144 configuraciones de la primera capa;
- `wr_reference.duckdb` y `simbad_negative.duckdb`;
- tres modelos representativos de la primera capa;
- exportaciones CSV compactas de los resultados de primera y segunda capa.

Los tres modelos de primera capa representan el perfil de ranking amplio, el líder en Average Precision y una configuración aceptada por los controles de estabilidad. La base de historia permite comparar las 144 configuraciones en la aplicación aunque el bundle no incluya los 144 archivos `.joblib`.

`training_history.duckdb` conserva las métricas resumidas del experimento de segunda capa, visibles en Validation Layers. Sus 64 modelos `.joblib` y los cuatro Parquet usados para ajustarlos quedaron fuera del bundle porque esa capa no mejoró el ranking operativo de primera capa.

La prediction pool completa sigue en construcción y queda fuera del bundle de revisión. La aplicación, las métricas y las predicciones de holdout pueden revisarse sin volver a entrenar ni ejecutar consultas masivas a Gaia.

2. Instalación

Se requiere Python 3.11. La instalación de revisión se realiza dentro de un entorno virtual y usa versiones bloqueadas con hashes. El lock conserva además las versiones de scikit-learn y XGBoost con las que se serializaron los modelos incluidos:

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --require-hashes -r requirements-review.txt
python -m pip install -e . --no-deps --no-build-isolation
```

Después se extrae el bundle sobre la misma raíz. El archivo `REVIEW_BUNDLE_MANIFEST.json` incluido en el ZIP registra la ruta, el tamaño y el SHA-256 de cada artefacto.

Las credenciales de Gaia, archivos `.env` y configuraciones personales no se distribuyen. Tampoco son necesarias para abrir la aplicación y revisar los entrenamientos incluidos.

Los archivos `.joblib` son objetos serializados de Python. Deben utilizarse únicamente desde la Release oficial del proyecto y después de verificar el SHA-256 publicado junto al ZIP.

3. Model Explorer

La aplicación Streamlit es la interfaz principal para revisar los resultados:

```
wr-detector explore-models --config configs/models.yaml
```

El comando abre <http://127.0.0.1:8501>. El launcher limita el servidor a la máquina local, mantiene activas las protecciones CORS/XSRF y desactiva la telemetría de uso de Streamlit. Dentro de la aplicación se selecciona el entrenamiento `run_v3_main` y, cuando corresponde, un `result_id`. Un `result_id` identifica una configuración concreta: dataset, features, algoritmo, sampler, hiperparámetros, métricas y artefactos.

Vista	Uso
Overview	Resumen del run, modelos destacados y selección del modelo activo.
Compare models	Comparación de AP, top-K, composición del holdout, FPR y estabilidad.
Model detail	PR/ROC, matriz de confusión, umbral, features y artefactos del modelo.
Case review	Revisión de WR recuperadas, contaminantes, falsos positivos y fuentes individuales.
Validation layers	Resumen del experimento de segunda capa y compatibilidad de lineage.

Cuadro 1: Vistas principales de Model Explorer.

Las tablas y gráficos se generan desde `data/databases/training_history.duckdb`. El modelo activo cambia únicamente cuando se confirma la selección en la interfaz; las filas marcadas en Compare controlan las comparaciones y no reemplazan el modelo activo.

4. Notebooks de auditoría

Los notebooks resumen verificaciones específicas y complementan la aplicación:

- `01_reference_and_color_locus.ipynb`: procedencia, cobertura fotométrica y locus;
- `02_model_selection_and_validation.ipynb`: split, métricas, modelos destacados y segunda capa;
- `03_prediction_pool_status.ipynb`: consulta, filtros y estado del build.

Los notebooks son artefactos de auditoría. La lógica de producción vive en `src/wr_detector` y en los archivos de `configs/`; editar una celda no modifica el pipeline.

5. Comandos reproducibles

Preparación de las muestras

```
wr-detector build-reference --config configs/reference.yaml
wr-detector audit-reference --db data/databases/wr_reference.duckdb
wr-detector export-reference-datasets --config configs/filters.yaml
wr-detector build-simbad-negative --config configs/simbad_negative.yaml
wr-detector export-simbad-negative-datasets --config configs/filters.yaml
wr-detector export-color-locus --config configs/filters.yaml
```

Estas etapas realizan consultas externas y no son necesarias para revisar el bundle. Los snapshots y hashes permiten verificar qué datos produjeron los exports disponibles.

Reducción y entrenamiento

```
wr-detector reduce-negatives --config configs/models.yaml
wr-detector train-models --config configs/models.yaml --run-id RUN_ID
wr-detector train-models --config configs/models.yaml '
  --run-id RUN_ID --resume-run
```

`-resume-run` conserva las configuraciones ya completadas. Los resultados se sincronizan en `data/databases/training_history.duckdb`. La segunda capa se conserva únicamente como un diagnóstico resumido en la base de historia.

Prediction pool

La consulta de producción sigue activa. Para inspeccionar sus contratos sin descargar datos:

```
wr-detector build-prediction-pool-exact-union '  
  --config configs/prediction_pool_exact_union.yaml --dry-run
```

Para auditar los tiles y manifiestos ya construidos:

```
wr-detector audit-prediction-pool-exact-union-build '  
  --config configs/prediction_pool_exact_union.yaml
```

El scoring completo comenzará cuando finalice la consulta y la auditoría del build resulte satisfactoria. El scorer aplica cada modelo solamente sobre fuentes que cumplen su locus, calidad fotométrica, requisito astrométrico y lista de features.

Pruebas

```
python -m pytest  
python -m compileall src/wr_detector
```

6. Ubicación de los artefactos

Contenido	Ruta
Historia de entrenamientos	<code>data/databases/training_history.duckdb</code>
Referencia WR	<code>data/databases/wr_reference.duckdb</code>
Muestra SIMBAD	<code>data/databases/simbad_negative.duckdb</code>
Runs y modelos	<code>reports/modeling/runs/{run_id}/</code>
Figuras públicas	<code>reports/public/figures/</code>
Bundle de revisión	GitHub Release de esta entrega

Cuadro 2: Rutas relevantes después de extraer el bundle.

Los directorios generados permanecen fuera de Git. El código y las configuraciones contienen la lógica reproducible; el bundle conserva la evidencia necesaria para revisar esta entrega.